# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- In many cases the Apriori candidate generate-and-test method significantly reduces the size of candidate sets, leading to good performance gain. However, it can suffer from two nontrivial costs.
  - *It may still need to generate a huge number of candidate sets*. For example, if there are $10^4$ frequent 1-itemsets, the Apriori algorithm will need to generate more than $10^7$ candidate 2-itemsets.
  - *It may need to repeatedly scan the whole database and check a large set of candidates by pattern matching*. It is costly to go over each transaction in the database to determine the support of the candidate itemsets.
- An interesting method in this attempt is called **frequent pattern growth (FP-growth)** which adopts a *divide-and-conquer* strategy as follows.



# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- Frequent Pattern Growth
  - It compresses the database representing frequent items into a **frequent pattern tree** (**FP-tree**) which retains the itemset association information.
  - It then divides the compressed database into a set of *conditional databases*, each associated with one itemset found so far, or “pattern fragment,” and mines each database separately.
  - For each “pattern fragment,” only its associated data sets need to be examined. Therefore this approach may substantially reduce the size of the data sets to be searched, along with the “growth” of patterns being examined.

## 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- **Example 4.5. FP-growth (finding frequent itemsets without candidate generation).** We reexamine the mining of transaction database, *D*, of Table 4.1.

**Table 4.1** A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

- The first scan of the database is the same as Apriori, which derives the set of frequent items (1-itemsets) and their support counts (frequencies).
- Let the minimum support count be 2.
- The set of frequent items is sorted in the order of descending support count. This resulting set or list is denoted by *L*.
  - Thus, we have $L = \{\{I_2: 7\}, \{I_1: 6\}, \{I_3: 6\}, \{I_4: 2\}, \{I_5: 2\}\}$.
- An FP-tree is then constructed as follows. First, create the root of the tree, labeled with "null." Scan database D a second time. The items in each transaction are processed in *L* order (i.e., sorted according to descending support count), and a branch is created for each transaction.



## 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

**Table 4.1** A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

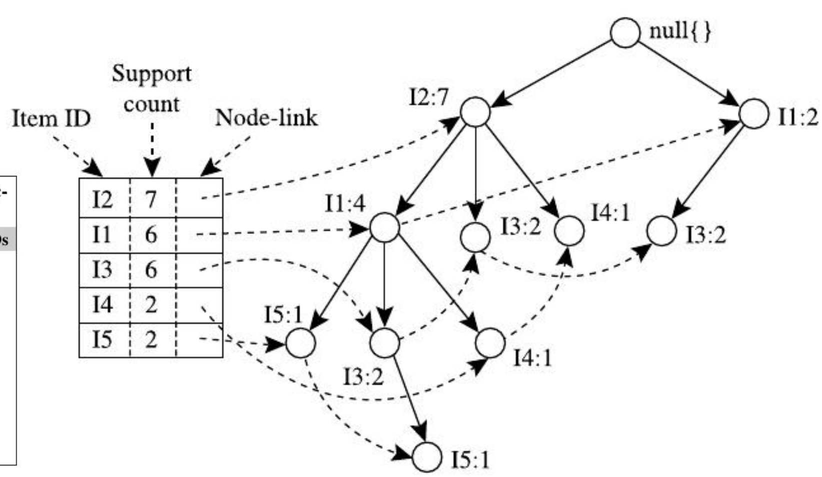


**FIGURE 4.7**
An FP-tree registers compressed frequent pattern information.

- The scan of the first transaction, "T100: I1, I2, I5," which contains three items (I2, I1, I5 in *L* order), leads to the construction of the first branch of the tree with three nodes, <I2: 1>, <I1: 1>, and <I5: 1> , where I2 is linked as a child to the root, I1 is linked to I2, and I5 is linked to I1.

# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

**Table 4.1** A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

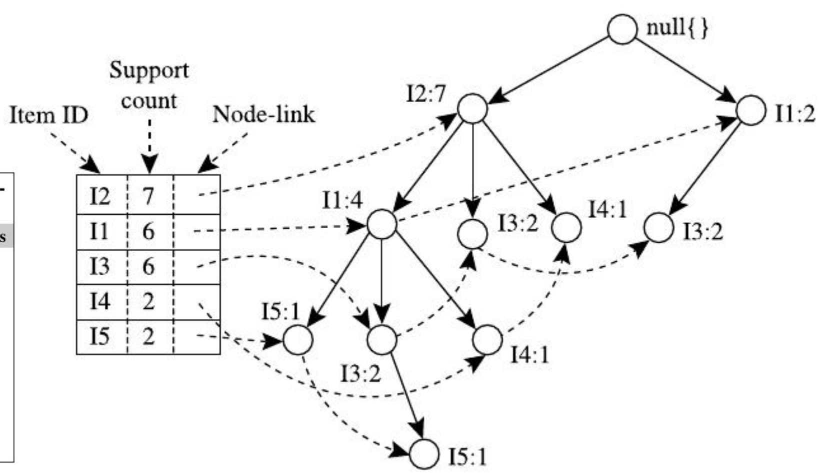


**FIGURE 4.7**

An FP-tree registers compressed frequent pattern information.

- The second transaction, T200, contains the items I2 and I4 in *L* order, which would result in a branch where I2 is linked to the root and I4 is linked to I2.
- However, this branch would share a common **prefix**, I2, with the existing path for T100. Therefore, we instead increment the count of the I2 node by 1, and create a new node, <I4: 1>, which is linked as a child to <I2: 2>.
- In general, when considering the branch to be added for a transaction, the count of each node along a common prefix is incremented by 1, and nodes for the items following the prefix are created and linked accordingly.




# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

**Table 4.1** A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |


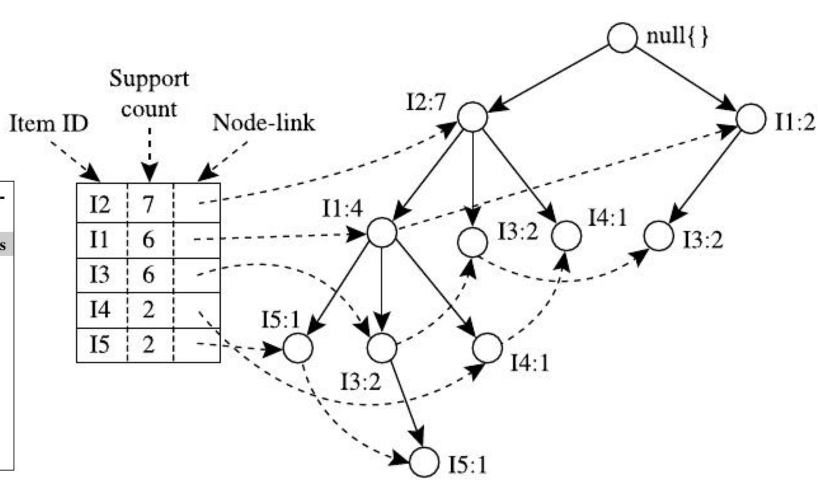



**FIGURE 4.7**

An FP-tree registers compressed frequent pattern information.


- To facilitate tree traversal, an item header table is built so that each item points to its occurrences in the tree via a chain of **node-links**.
- The problem of mining frequent patterns in databases is transformed into that of mining the FP-tree.

# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

Table 4.1 A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

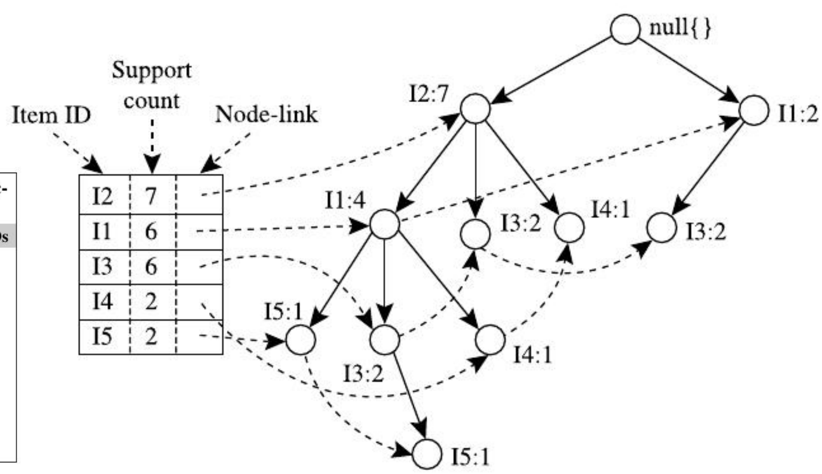


**FIGURE 4.7**

An FP-tree registers compressed frequent pattern information.

- The FP-tree is mined as follows. Start from each frequent length-1 pattern (as an initial **suffix pattern**), construct its **conditional pattern base** (a "subdatabase," which consists of the set of *prefix paths* in the FP-tree cooccurring with the suffix pattern), then construct its (*conditional*) FP-tree, and perform mining recursively on the tree. The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.



# 2. Frequent Itemset Mining Methods
## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

Table 4.1 A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

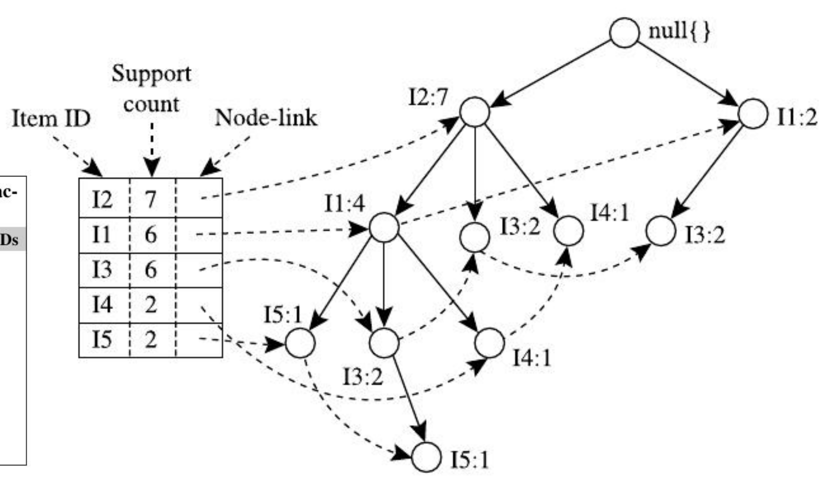


**FIGURE 4.7**

An FP-tree registers compressed frequent pattern information.

- We first consider I5, which is the last item in *L*, rather than the first.
- I5: Suffix
  - Two prefix paths: <I2, I1, I5: 1>, <I2, I1, I3, I5: 1>, which form its conditional pattern base.
  - Using this conditional pattern base as a transaction database, we build an I5-conditional FP-tree, which contains only a single path, <I2: 2, I1: 2>; I3 is not included because its support count of 1 is less than the minimum support count.
  - Combinations of frequent patterns: {I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}.

# 2. Frequent Itemset Mining Methods

## 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

**Table 4.1** A transactional data set.

| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

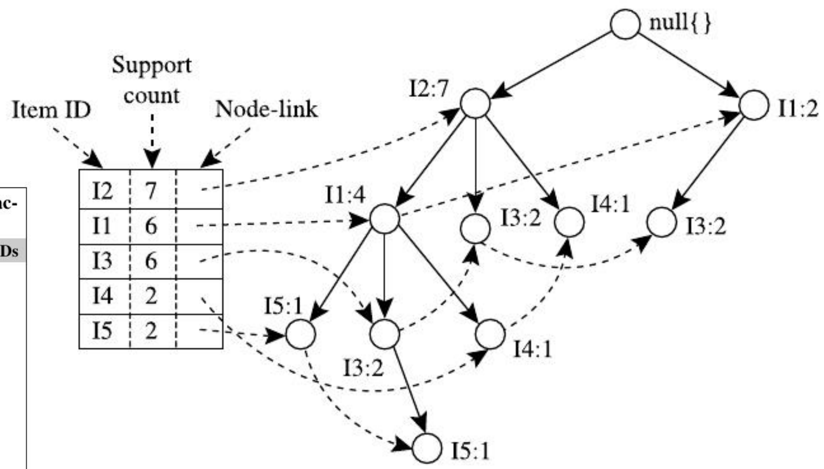


**FIGURE 4.7**

An FP-tree registers compressed frequent pattern

**Table 4.2** Mining the FP-tree by creating conditional (sub-)pattern bases.

| Item | Conditional Pattern Base | Conditional FP-tree | Frequent Patterns Generated |
|---|---|---|---|
| I5 | {{I2, I1: 1}, {I2, I1, I3: 1}} | ⟨I2: 2, I1: 2⟩ | {I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2} |
| I4 | {{I2, I1: 1}, {I2: 1}} | ⟨I2: 2⟩ | {I2, I4: 2} |
| I3 | {{I2, I1: 2}, {I2: 2}, {I1: 2}} | ⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩ | {I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2} |
| I1 | {{I2: 4}} | ⟨I2: 4⟩ | {I2, I1: 4} |